

Eventia

Épica 2: Gestión de Eventos e Inscripciones (HU03, HU04)

Grupo 1 - Ingeniería de Software II

Fecha: 5 de septiembre de 2026

1. Alcance de esta entrega

Se implementaron, dentro de `backend/`, las tareas de la Épica 2 correspondientes a las historias **HU03 (Creación y Parametrización de Eventos)** y **HU04 (Inscripción de Participante a un Evento)**:

Tarea	Descripción	Estado
3.2	<code>POST /api/v1/eventos</code> con validación de fechas ($\text{fin} \geq \text{inicio}$) y cupo positivo	Hecho
3.3	<code>GET /api/v1/eventos</code> (con filtros) y <code>PUT /api/v1/eventos/{id}</code>	Hecho
4.2	<code>POST /api/v1/inscripciones</code> con lógica transaccional (duplicado, cupo, estado)	Hecho
4.3	Derivación automática a "Lista de Espera" según la política del evento	Hecho

Las tareas de base de datos **3.1** y **4.1** (diseño de tablas `TIPO_EVENTO`, `EVENTO`, `POLITICA_INSCRIPCION`, `PARTICIPANTE`) no estaban asignadas a esta entrega, pero fue necesario modelarlas a nivel de ORM (SQLAlchemy) porque los endpoints de negocio dependen de ellas. Ver sección 6 (Notas y pendientes).

2. Historias de usuario cubiertas

2.1 HU03 — Creación y Parametrización de Eventos

Descripción: Como Organizador, quiero crear un nuevo evento indicando su tipo, rango de fechas, lugar y cupo máximo para publicarlo y habilitar inscripciones.

Criterios de aceptación:

- Dado un formulario de alta de evento, cuando el organizador completa los campos obligatorios (tipo de evento, fechas válidas, cupo > 0), entonces el evento se guarda en estado "Borrador" o "Publicado".
- Dado un evento con fecha de fin anterior a la fecha de inicio, cuando se intenta guardar, entonces el sistema muestra un mensaje de error de validación.

2.2 HU04 — Inscripción de Participante a un Evento

Descripción: Como Participante, quiero inscribirme a un evento disponible para asegurar mi cupo y recibir mi comprobante.

Criterios de aceptación:

- Dado un evento con cupo disponible, cuando un participante envía su formulario de inscripción, entonces el sistema crea el registro con estado "Confirmada" y reduce el cupo

disponible.

- Dado que un participante ya posee una inscripción activa en el mismo evento, cuando intenta inscribirse nuevamente, entonces el sistema bloquea la operación indicando que ya está registrado.
- Dado un evento sin cupos pero con la política "permite lista de espera" activa, cuando ingresa una nueva inscripción, entonces se registra en estado "Lista de Espera".

3. Modelo de datos agregado

Se agregaron al ORM (`app/models.py`) las entidades necesarias para soportar ambas historias:

- **TipoEvento** : catálogo parametrizable de tipos de evento (congreso, seminario, taller, jornada).
- **Evento** : tipo, nombre, descripción, fecha de inicio/fin, lugar, cupo máximo y estado (borrador / publicado).
- **PoliticaInscripcion** : relación 1 a 1 con **Evento** ; define `permite_lista_espera`, `requiere_aprobacion`, `fecha_limite` y `min_asistencia_certificado` (con valores por defecto si no se especifican).
- **Participante** : datos de contacto de la persona que se inscribe (documento, nombres, apellidos, email único, institución). No requiere una cuenta de usuario del sistema.
- **Inscripcion** (extendida): ahora incluye `id_evento` e `id_participante`, con una restricción **UNIQUE** (`id_evento`, `id_participante`) que impide que un mismo participante quede inscripto dos veces al mismo evento.

4. Endpoints implementados

4.1 `app/api/eventos.py`

POST <code>/api/v1/eventos</code>	Crea un evento (requiere rol "organizador")
GET <code>/api/v1/eventos</code>	Lista eventos con filtros (público)
PUT <code>/api/v1/eventos/{evento_id}</code>	Actualiza un evento (requiere rol "organizador")

- **Validaciones al crear/actualizar:** el `id_tipo_evento` debe existir (404 si no), `fecha_fin` no puede ser anterior a `fecha_inicio` (422), `cupos_maximo` debe ser mayor a 0 (validado por el schema).
- **Filtros del listado:** `id_tipo_evento`, `estado`, `q` (búsqueda por nombre), `fecha_desde` y `fecha_hasta` (eventos cuyo rango de fechas se solapa con el filtro).
- La política de inscripción se puede enviar anidada en el body (`politica`); si no se envía, se crea con valores por defecto (`permite_lista_espera=false`, `min_asistencia_certificado=75`).

4.2 app/api/inscripciones.py

POST /api/v1/inscripciones
(público)

Inscribe a un participante a un evento

Lógica transaccional del endpoint:

1. Busca el evento (con bloqueo de fila `SELECT ... FOR UPDATE` para evitar condiciones de carrera al validar el cupo) y el rol de participante; responde `404` si no existen.
2. Busca un `Participante` existente por email (sin distinguir mayúsculas) o crea uno nuevo con los datos recibidos.
3. Si ya existe una inscripción de ese participante a ese evento, responde `409 Conflict`.
4. Cuenta las inscripciones con estado `confirmada` para el evento:
 - Si hay cupo disponible → estado `confirmada`.
 - Si no hay cupo pero `politica.permite_lista_espera` es verdadero → estado `lista_de_espera`.
 - Si no hay cupo y no se permite lista de espera → `409 Conflict`.
5. Persiste la inscripción con `fecha_inscripcion` (UTC) y la devuelve.

5. Estructura de archivos actualizada

```
backend/
├── app/
│   ├── api/
│   │   ├── auth.py
│   │   ├── roles.py
│   │   ├── users.py
│   │   ├── eventos.py          # nuevo: HU03 (tareas 3.2, 3.3)
│   │   └── inscripciones.py    # nuevo: HU04 (tareas 4.2, 4.3)
│   ├── core/
│   ├── db.py
│   ├── models.py              # + TipoEvento, Evento, PoliticaInscripcion,
│   │                           #   Participante; Inscripcion extendida
│   ├── schemas.py              # + EventoCreate/Update/Response,
│   │                           #   PoliticaInscripcionData,
│   │                           #   InscripcionCreate/Response
│   └── main.py                 # routers de eventos e inscripciones registrados
├── tests/
│   ├── test_auth.py
│   ├── test_role_integrity.py  # actualizado (Inscripcion ahora exige
│   │                           evento/participante)
│   ├── test_eventos.py         # nuevo
│   └── test_inscripciones.py   # nuevo
```

6. Notas y pendientes

sql/ desactualizado respecto al ORM: los scripts de migración en `sql/` usan nombres de columna distintos a los del modelo real (`correo/contrasena_hash` vs. `email/password_hash` en `usuario`, por ejemplo) y no incluyen aún las tablas de esta época (`evento`, `tipo_evento`, `politica_inscripcion`, `participante`, ni la `inscripcion extendida`). Los tests corren contra un esquema generado directamente desde `app/models.py` (SQLite en memoria), que es la fuente de verdad actual del backend. Falta sincronizar o generar los scripts SQL correspondientes (tareas 3.1 y 4.1) para que el despliegue en PostgreSQL coincida con el ORM.

- **Rol "organizador":** no viene seedeado en ningún script; un administrador debe crearlo mediante `POST /api/v1/roles/sistema`, igual que cualquier otro rol parametrizable del sistema.
- **Estado `lista_de_espera`:** se agregó a la lista de estados de inscripción considerados "activos" en `roles.py`, para que no se pueda borrar un rol de participante que tenga inscripciones en espera.

7. Validaciones realizadas

- Se corrió la suite completa de `pytest`: **19 tests pasan** (incluye los 2 preexistentes de auth, 2 de integridad de roles y los 15 nuevos de eventos e inscripciones).
- Se verificó el arranque de la aplicación FastAPI y el registro correcto de las nuevas rutas en el esquema OpenAPI (`/api/v1/eventos`, `/api/v1/eventos/{evento_id}`, `/api/v1/inscripciones`).
- Casos cubiertos por los tests nuevos:
 - Creación de evento con estado por defecto "borrador" y política por defecto.
 - Rechazo de evento con fecha de fin anterior a la de inicio.
 - Rechazo de evento con tipo de evento inexistente.
 - Filtro de listado de eventos por estado.
 - Actualización de evento: validación de fechas resultantes y actualización de la política.
 - Inscripción confirmada cuando hay cupo disponible.
 - Rechazo de inscripción duplicada del mismo participante al mismo evento.
 - Paso a lista de espera cuando se agota el cupo y la política lo permite.
 - Rechazo de inscripción sin cupo ni lista de espera habilitada.
 - Reutilización del mismo `Participante` al inscribirse a distintos eventos con el mismo email.